Latest updates: https://dl.acm.org/doi/10.1145/3664476.3664515

RESEARCH-ARTICLE

GNN-IDS: Graph Neural Network based Intrusion Detection System

ZHENLU SUN, Uppsala University, Uppsala, Uppsala, Sweden

ANDRÉ M.H. TEIXEIRA, Uppsala University, Uppsala, Uppsala, Sweden

SALMAN TOOR, Uppsala University, Uppsala, Uppsala, Sweden

Open Access Support provided by:

Uppsala University

PDF Download
3664476.3664515.pdf
06 April 2026
Total Citations: 42
Total Downloads: 6334

Published: 30 July 2024

Citation in BibTeX format

ARES 2024: The 19th International Conference on Availability, Reliability and Security
July 30 - August 2, 2024
Vienna, Austria

ARES '24: Proceedings of the 19th International Conference on Availability, Reliability and Security (July 2024)
https://doi.org/10.1145/3664476.3664515
ISBN: 9798400717185

GNN-IDS: Graph Neural Network based Intrusion Detection System

Zhenlu Sun
zhenlu.sun@it.uu.se
Uppsala University
Uppsala, Sweden

André M. H. Teixeira
andre.teixeira@it.uu.se
Uppsala University
Uppsala, Sweden

Salman Toor
salman.toor@it.uu.se
Uppsala University
Uppsala, Sweden

ABSTRACT

Intrusion detection systems (IDSs) are widely used to identify anomalies in computer networks and raise alarms on intrusive behaviors. ML-based IDSs generally take network traces or host logs as input to extract patterns from individual samples, whereas the inter-dependencies of network are often not captured and learned, which may result in large amounts of uncertain predictions, false positives, and false negatives. To tackle the challenges in intrusion detection, we propose a graph neural network-based intrusion detection system (GNN-IDS), which is data-driven and machine learning-empowered. In our proposed GNN-IDS, the attack graph and real-time measurements that represent static and dynamic attributes of computer networks, respectively, are incorporated and associated to represent complex computer networks. Graph neural networks are employed as the inference engine for intrusion detection. By learning network connectivity, graph neural networks can quantify the importance of neighboring nodes and node features to make more reliable predictions. Furthermore, by incorporating an attack graph, GNN-IDS could not only detect anomalies but also identify the malicious actions causing the anomalies. The experimental results on a use case network with two synthetic datasets (one generated from public IDS data) show that the proposed GNN-IDS achieves good performance. The results are analyzed from the aspects of uncertainty, explainability, and robustness.

CCS CONCEPTS

• **Security and privacy** → **Intrusion/anomaly detection and malware mitigation**; **Intrusion detection systems**;

KEYWORDS

Intrusion Detection System, Graph Neural Network, Uncertainty, Explainability, Robustness

ACM Reference Format:
Zhenlu Sun, André M. H. Teixeira, and Salman Toor. 2024. GNN-IDS: Graph Neural Network based Intrusion Detection System. In *The 19th International Conference on Availability, Reliability and Security (ARES 2024), July 30–August 02, 2024, Vienna, Austria.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3664476.3664515

This work is licensed under a Creative Commons Attribution International 4.0 License.

ARES 2024, July 30–August 02, 2024, Vienna, Austria
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1718-5/24/07
https://doi.org/10.1145/3664476.3664515

1 INTRODUCTION

In recent times, there has been a notable surge in cyber attacks targeting critical infrastructures, including finance transaction platforms, public transportation systems, healthcare data centers, and more. In 2021, the Colonial oil pipeline suffered a severe targeted ransomware attack [13]. More recently, KillNet orchestrated unprecedented DDoS strikes that successfully incapacitated Lithuania's power grid. Their impact extended to over a dozen U.S. airports, leading to canceled flights and disruptions for operators [6]. These attacks have led to substantial economic and societal losses. Therefore, it has become imperative to adeptly identify intrusions and safeguard crucial assets from malicious activities.

An Intrusion Detection System (IDS) is such a system that can identify intrusive behaviors and raise alerts indicating intrusions are happening in computer networks [5, 35]. From the perspective of approaches adopted for intrusion detection, IDSs are broadly classified into two categories, i.e., Signature-based IDS (SIDS) and Anomaly-based IDS (AIDS). A SIDS identifies attacks by matching the patterns in a pre-defined knowledge base. SIDS can effectively identify the anomalies whose patterns have been stored in the knowledge base. However, for unseen attacks such as zero-day attacks, SIDS cannot make an accurate detection [19]. An AIDS built on statistics, expert knowledge, or machine learning (ML) can identify unseen anomalous behaviors, overcoming the limitation of SIDS. Among AIDS, ML-based IDSs [20, 34, 38] stand out, characterized by intelligently extracting patterns and identifying anomalies from large amounts of data. Facing complex scenarios, ML-based AIDS may falsely predict unseen normal behaviors as intrusions, resulting in false positive alerts that cost much manual analysis by security experts [18]. One common limitation of these ML-based AIDS is that each data sample is treated independently, while the inter-dependencies of a certain network are not considered.

A graph is a well-formatted data structure that can represent and extract complex patterns of data [21]. Characterized by powerful data representation, expressive and learnable inter-dependencies [1, 17], graphs are widely used in social media, chemistry, biology, computer communication, etc. Recently, Graph Neural Networks (GNNs) have been gaining more attentions due to the ability to effectively learn the hidden representation of node and topological information of graph-structured data [27, 42]. More recently, some works have also been done to study the explainability of GNNs [39, 43]. In the field of cybersecurity, there have been a few GNN-related approaches [24, 28, 31] that show successes in detecting anomalies in computer networks by building graphs upon network traces or host logs captured in networks. Nevertheless, these approaches are highly dependent on public datasets, which could limit the construction and representation of graphs, without revealing the

complete attack scenarios. Moreover, the abundant information in computer systems or networks such as vulnerabilities is not fully utilized to train GNN models, which could to some degree make the GNN models' predictions uncertain and unexplainable, resulting in false positives and false negatives. Additionally, as hackers are becoming more sophisticated by equipping with advanced attacking skills, highly robust IDSs to adversarial examples should be designed to avoid detection evasions [29].

Attack graph [4] is a modelling framework that depicts correlated multi-stage attacks in computer systems and networks and can be used to analyze potential attack paths that an adversary could take by exploiting vulnerabilities in networks. Essentially, attack graph and real-time measurements are separate for different research purposes. To address the problems in intrusion detection, this paper proposes a Graph Neural Network based Intrusion Detection System (GNN-IDS) in which the attack graph is adopted to represent the structural information and static attributes of computer networks, real-time measurements are used to simulate the dynamics in networks, and Graph Neural Networks are employed as the inference engine for intrusion detection. To apply the attack graph in GNN-IDS, we first encode the textual representation of an attack graph using one-hot encoding. For the construction of input graphs to GNN models, we propose to associate the encoded attack graph with real-time measurements so that the static and dynamic attributes of networks, as well as the inter-dependencies, could be included in the input graphs. Empowered by GNN models with well-designed architectures, GNN-IDS could not only detect which node at a certain time point is under attack, but also identify which action is taken by attackers to realize the attack.

For evaluation, three GNN models, i.e., Graph Convolution Network (GCN) [22], Graph Convolution Network with learnable Edge Weights (GCN-EW), and Graph Attention Network (GAT) [37], in which the inter-dependencies are not learned, implicitly learned, and explicitly learned, respectively, are developed. We create two synthetic datasets (one from public CIC-IDS2017 [32]) for a use case network to validate the effectiveness and efficiency of GNN-IDS. The results are analyzed from the aspects of uncertainty, explainability, and robustness. It turns out that our proposed GNN-IDS achieves good performance in regards to standard metrics such as precision, F1-score, recall. The explainability of GNN-IDS is showcased assisted by *GNNExplainer* [43]. Lastly, the high robustness of GNN-IDS against adversarial attacks is also presented. With the experimental results, we would like to answer the following research questions:

- **RQ1**: Can GNN models within the GNN-IDS provide more confident predictions?
- **RQ2**: Can the predictions of GNN-IDS be explained?
- **RQ3**: Is the GNN-IDS robust to evolving networks?

In summary, the main contributions of our work are three-fold:

- A graph neural network based intrusion detection system [1] is proposed and implemented in which attack graph and real-time measurements are incorporated to represent the static and dynamic information in computer networks, and graph neural networks are leveraged as the inference engine for intrusion detection.
- We innovatively associate the separate attack graph and real-time measurements to construct the input graphs to GNN models.
- We address the challenges in intrusion detection from three aspects, i.e., uncertainty, explainability, and robustness, which are elaborated with the predictions of our proposed GNN-IDS.

The rest of the paper is organized as follows. In section 2, we review the related works. In Section 3, we present our proposed GNN-IDS in detail. In section 4, a use case network and two synthetic datasets are introduced. In section 5, the experimental results are presented and analyzed. In section 6, conclusions are made based on the experimental results, and future works are discussed.

2 RELATED WORKS

In this section, we broadly review the state-of-the-art graph-based approaches for intrusion or anomaly detection, as our proposed GNN-IDS is also highly related to graphs, i.e., attack graph, graph-structured data, and graph neural networks.

[31] proposes a graph representation to capture the structural flow patterns of attacks. They translate common attacks into host-connect graphs to represent attacks with different flows, for which a particular GNN model is designed. Evaluation on the CIC-IDS2017 dataset shows that it achieves a weighted F1-score of 0.99 which is comparable to state-of-the-art ML models including multi-layer perceptron, Ada-boost, Random Forest, and ID3. With varying flow-level features, the high accuracy of the GNN model remains, indicating its good robustness against adversarial attacks.

To detect anomalies in multivariate time series data, an attention-based Graph Deviation Network (GDN) was proposed in [9]. It learns the relationships between sensors and detects deviations from relationship patterns. Evaluation on two real-world water system datasets shows that GDN outperforms baseline models in precision, recall, and F1-score. The interpretability of the GDN is presented with its sensor embedding. The learned attention weights are used to localize anomalies. GDN achieves high precision, while recall and F1-score metrics needs to be further improved.

In [28], E-GraphSAGE, a GNN model that can learn flow-based features and topology patterns of network flows, is developed for intrusion detection for IoT. They use network flows to construct network graphs in which nodes represent the source or destination, and edges represent the network flows. A GNN model is designed and evaluated on four public datasets. The results show that E-GraphSAGE achieves comparable or in most cases outperforms state-of-the-art ML-based approaches, for both binary and multi-class classification tasks. The consistent results on the four datasets demonstrate its robustness and generality.

The work [24] presents an unsupervised graph mining approach, BotGM, to detect the botnets in traffic flow. BotGM relies on NetFlow data to model the dependencies between flows. Evaluations on the CTU-13 dataset [12] show that BotGM can extract abnormal graph candidates on specific samples, achieving an accuracy of around 90% in multiple cases. Compared to a few baselines, BotGM achieves an accuracy of 95% at the graph level while other baselines only achieve 50% at the single flow level.

In [10], the authors propose a novel flow-based graph structure, in which nodes represent network flows and edges link two connected flows. This graph structure provides topological information

[1]The code is available in https://github.com/zhenlus/GNN-IDS.git.

such as attackers' behavioral patterns, iterative malicious behaviors, sequential and parallel attacks. The flows' features are embedded and fed into GNN layers to extract topological information, and then forwarded to a decision-making module to detect attacks. Evaluations on two public datasets show that it outperforms other ML-based and Graph-based solutions.

A real-time unsupervised ML based malicious traffic detection system, HyperVision, is presented in [11]. In HyperVision, the flow interaction graph is built upon network flows to capture the interaction patterns among different flows. The core component is a four-step lightweight unsupervised graph learning method which can detect the anomalous traffic for both encrypted and unencrypted traffic. The accuracy and efficiency of HyperVision are validated by replaying 92 attack datasets with 48 encrypted malicious traffic. The experimental results show it achieves great improvements at accuracy and F1-score over five state-of-the-art methods. The advantages of HyperVision are also presented from the aspects of throughput, latency, and resource consumption.

As argued in [7], future IDS should track how attacks evolve across system layers by adopting attack graphs to overcome the limitations of current IDS that are unable to identify unseen attacks and access particular programs to confirm attacks. An attack graph based intrusion detection approach is proposed and a few challenges to the realization of their proposal are identified. With two use cases, they showcase how to apply their approach. Overall, a conceptual overview of why and how attack graphs should be used for the next generation of IDS is presented. However, the concept is not implemented and the challenges remain.

Apart from the works above, more studies have been done for intrusion or anomaly detection using GNNs [15, 23, 25, 41]. In [23], a supervised GNN model is proposed based on the Activity and Event Network (AEN), and achieves an accuracy score of 76% on a DDoS dataset and 88% on a TOR-nonTOR dataset. [25] proposes a GNN and transformer based model to detect anomalies in events in BETH dataset, which achieves around 8% of improvement. In [15], a GCN model is evaluated on an insider threat dataset, and it shows large improvements over RF, LR, SVM, and CNN models. In [41], a GNN-empowered solution is demonstrated in three cases in smart transportation, smart energy, and smart factory.

In contrast to the above studies, our approach associates the separate attack graph and real-time measurements to construct graphs that represent evolving computer networks, and employs graph neural networks to learn network structural information and node attributes. With the well-defined architecture of GNN models and appropriate association of the attack graph and real-time measurements, the GNN models can learn complete attack scenarios and make more confident predictions that are aware of the attack scenarios. To the best of our knowledge, our proposed GNN-IDS is the first implementation of such an intrusion detection system that incorporates the attack graph, real-time measurements, and graph neural network for intrusion detection.

3 SYSTEM FRAMEWORK

In this section, we present the proposed graph neural network based intrusion detection system (GNN-IDS) whose framework is depicted in Figure 1. We first introduce the main components of GNN-IDS, i.e., computer networks, attack graph, real-time measurements, graph neural networks, and inference. Next, we summarize the flow to deploy the GNN-IDS in real-world systems.

3.1 Computer Networks

The computer networks investigated in this paper primarily pertain to physical Information Technology (IT) systems. This category of IT systems span a spectrum ranging from small-sized compute and storage platforms to IT sectors in critical infrastructures. The Cyber-Physical System (CPS) and Internet of Things (IoT), characterized by physical sensors, actuators, and plants, are not within the purview of our examination.

Network Representation. IT networks are generally composed of host machines, network configurations, assets, and vulnerabilities. Host machines, installed with physical compute and storage devices, are the basis of computer networks. Network configurations refer to the setups of switch, firewall, and communication channels (such as TCP, UDP, FTP, and SMTP protocols). Assets represent critical services, software applications, or databases that should not be interrupted but potential to be attacked. Vulnerabilities refer to flaws in assets resulting from a weakness that can be exploited, causing a negative impact on the confidentiality, integrity, or availability of networks.

In a nutshell, the proposed GNN-IDS is tailored to IT networks rather than CPS or IoT. The advantages inherent to the application of GNN-IDS in IT systems are, on one hand IT systems are convenient to represent with attack graphs, on the other hand the measurements of IT systems are easy to capture with network monitoring tools, which will be elaborated in sections 3.2 and 3.3.

3.2 Attack Graph

An attack graph is a directed acyclic graph that represents the potential attack paths that attackers could take by exploiting the vulnerabilities in networks [26]. Relying on the aforementioned network information, numerous attack graphs [2, 14, 16, 30, 33] have been proposed, among which the multi-host, multi-stage vulnerability analysis (MulVAL) [30] stands out featuring diverse extensions, low complexity, and high community activeness, and is therefore used for attack graph generation in our proposed GNN-IDS.

Attack graph representation. Attack graphs generated by MulVAL contain fact nodes, action nodes, privilege nodes, and logical dependencies among nodes. The vulnerability information in networks which is valuable to security analysis is also stored in the attack graphs. The abstract diagram of such attack graphs is shown in Figure 2. The three types of nodes are summarized as follows:

- Fact node: in rectangle, labeled with a statement starting with a predicate followed by arguments, represents the network information, e.g., hosts, assets, network configurations, and vulnerabilities.
- Action node: in oval, represents interaction rules that specify how attackers could compromise the network.
- Privilege node: in diamond, represents compromises of assets in networks by code execution, file access, privilege escalation, etc.

Attack graph encoding. Generally, a graph can be expressed in the form $\mathcal{G} = (V, E)$, where $v \in V$ refers to vertices or nodes and

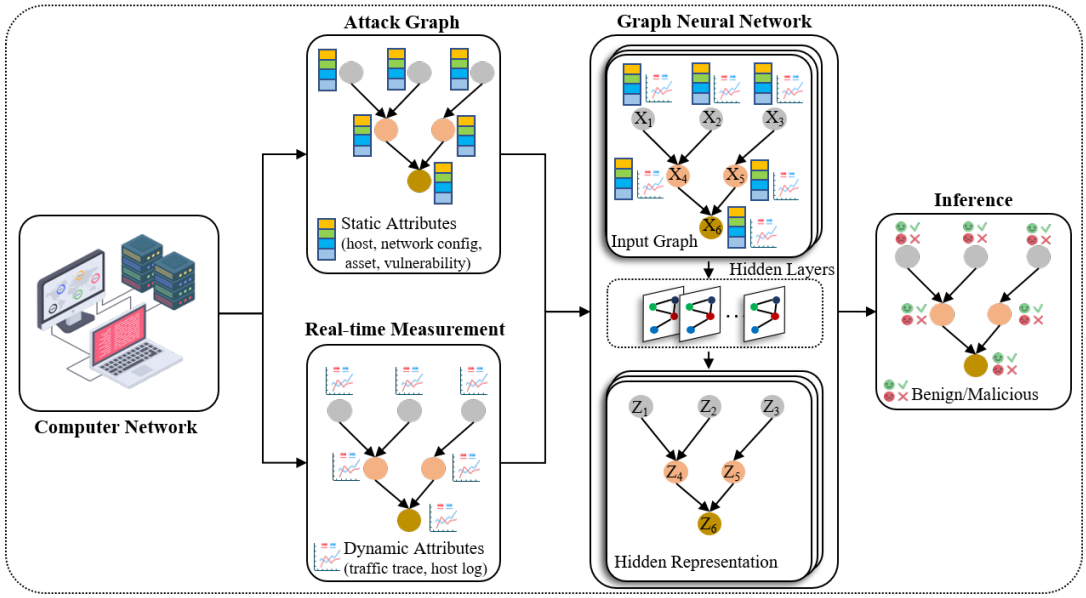


Figure 1: System framework of the proposed graph neural network based intrusion detection system (GNN-IDS).

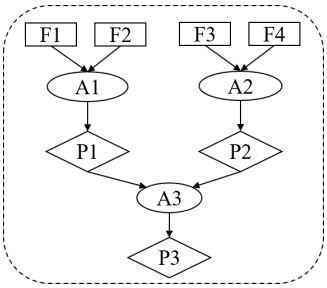


Figure 2: Diagram of attack graph generated with MulVAL toolkit. Vulnerability information is stored in fact nodes ('F*': Fact node; 'A*': Action node; 'P*': Privilege node).

$e_{ij} = (i, j) \in E$ are directed links or edges pointing from source node v_i to destination node v_j. To encode the attack graph generated by MulVAL, we hereby introduce the notations:

- $C = \{w_i\}_{i=0}^{D-1}$, where w_i is a certain token, refers to the built corpus containing D tokens.
- S_v, refers to the statement of node v in string format.
- $\mathbf{f}_v \in \mathbb{R}^D$, is the encoded feature vector of S_v. Element f_{vk} in $\mathbf{f}_v$ is obtained by:

$$f_{vk} = \begin{cases} 1, & \text{if } w_k \in S_v \\ 0, & \text{otherwise} \end{cases} \tag{1}$$

With the definitions above, an attack graph generated by MulVAL can then be represented as $\mathcal{G} = (V, E, \mathbf{F})$ in which $\mathbf{F} \in \mathbb{R}^{n \times D}$ represents the static attributes of all nodes in the attack graph. The pseudo-code for encoding the attack graph generated by MulVAL toolkit is shown in Algorithm 1.

Algorithm 1 Pseudo-code for encoding attack graph

Input: nodes V, edges E, statements $\{S_v | \forall v \in V\}$
Output: encoded attack graph $\mathcal{G} = (V, E, \mathbf{F})$

```
i ⇐ 0
for v ∈ V do
    for w ∈ S_v do
        if w ∉ C then
            C_i ⇐ w
            i ⇐ i + 1
        end if
    end for
end for
D ⇐ |C|
for v ∈ V do
    i ⇐ 0
    while i < D do
        if C_i ∈ S_v then
            f_vi ⇐ 1
        else
            f_vi ⇐ 0
        end if
        i ⇐ i + 1
    end while
end for
```

3.3 Real-time Measurements

The real-time measurements, which are usually in time series format, are crucial for dynamic analysis of complex computer networks. To prepare the dataset for GNN-IDS, we use two kinds of data sources, i.e., synthetic data and public data, to simulate the benign and attack scenarios in networks.

3.3.1 Synthetic Data. Cybersecurity remains the most sensitive domain, due to the concerns of security and privacy, and few organizations are willing to disclose their cybersecurity data. Most of the publicly available cybersecurity datasets are collected on special kinds of networks where plenty of network traffic records for benign and malicious behaviors are generated and captured. In many cases, infrastructure information needed in our approach is not or partially open-sourced. One feasible solution is synthetic

data, which provides the maximum degree of freedom and control over what can be featured and how the data is distributed.

Assumptions. To realize more realistic simulations of benign and attack scenarios in networks, we make a few assumptions below for synthetic data generation.

- **A1:** Some measurements follow certain probability distributions, e.g., Gaussian Distribution and Poisson Distribution, which are commonly seen in practice and studied in [3] to normalize raw data. Other measurements whose distribution does not rigorously follow certain probability distributions, can be expressed as nonlinear combinations of measurements that follow probability distributions.
- **A2:** For benign and malicious behaviors, each measurement follows the same family of probability distributions but with varying parameters, so that they have different patterns.
- **A3:** Persistent attack actions such as executing a code impact measurements that follow Gaussian distribution, and non-persistent attack actions such as accessing a network or file, impact measurements that follow Poisson distribution.
- **A4:** Measurements are captured per action in the network, as action-level measurements are consistent with the attack scenarios in attack graph and can be easily associated with attack graph.
- **A5:** At the same time, only one asset in the network can be attacked as attackers usually conduct sequential actions to compromise assets in real-world systems.

Definition. At time t, the real-time measurements of a network can be expressed in the form $\mathbf{M}^t = \{\mathbf{m}^t_{action}, \forall action \in actions\}$ where $\mathbf{m}^t_{action} \in \mathbb{R}^K$ and K is the number of measured variables. Each measurement could also be represented with the aggregated result within a certain time window, i.e., the k-th measurement at time t can be obtained with the formulation:

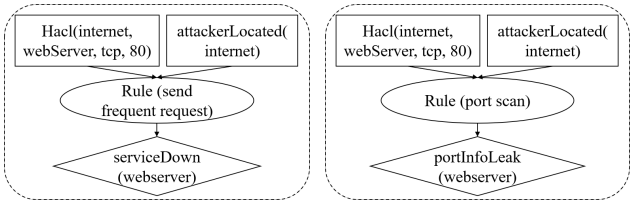
$$m^t_{action,k} = AGG(m^{t_1}_{action,k}, \ldots, m^{t_2}_{action,k}),\ t_1 \leq t \leq t_2 \quad (2)$$

where the aggregation operator AGG could be max, min, average, or custom ones as per specific requirements.

It is worth noting that, with the above assumptions, we purposely expect the machine learning model to extract patterns from normal and abnormal data to identify anomalies, instead of representing an accurate and correct characterization of real-world measurements.

3.3.2 Public Data. Compared to synthetic data, public data is more commonly used in academic research and industrial applications, as they are collected from realistic networks/testbeds and therefore more trustworthy. To make it more realistic, we also attempt to borrow the network traces from CIC-IDS2017 [32], one of the most widely used public datasets, to construct input graphs to GNN-IDS.

The limitations of CIC-IDS2017 are explicitly seen when constructing the input graphs with its traffic traces. Firstly, for almost all attack scenarios conducted in their experiments, the victim host remains the same, and so does the attack host, indicating low network complexity. Secondly, the scenarios in CIC-IDS2017 do not reveal multi-step attacks. Figure 3 depicts the attack paths of two attack scenarios (DoS and PortScan) in CIC-IDS2017, from which we observe that both attacks start from the attack host and end at the victim *webServer*, and only reveal one-step attack.



(a) DoS attack **(b) PortScan attack**

Figure 3: Diagram for the attack paths of DoS attack and PortScan attack in CIC-IDS2017.

Above all, the network structural information in CIC-IDS2017 cannot be directly used in our approach. To suit CIC-IDS2017 with the proposed GNN-IDS, we propose to relate its individual traces to a network with a longer attack path, such as the use case network introduced in section 4.1. We assume different attack scenarios in CIC-IDS2017 correspond to different attack actions in attack graph. For example, accessing a network and executing a code are related to DoS attack and PortScan attack in CIC-IDS2017, respectively. The specific association between actions in attack graph and traces in CIC-IDS2017 is introduced in section 4.2.

3.4 Graph Neural Networks

We first summarize the mechanism of graph neural networks. Next, we introduce the two graph operators, graph convolutional operator and graph attentional operator, which are the foundations of GCN, GCN-EW and GAT models in the proposed GNN-IDS. Lastly, we discuss the construction of input graphs to GNN-IDS.

Graph neural network. Nowadays have seen rising attention gained by graph neural networks [42] which are graph-structured data oriented, capable of learning the network topology and relationships among nodes. GNNs can be applied to different-level tasks such as node-level prediction, edge-level prediction, and graph-level prediction. The intrusion detection problem studied in this paper is narrowed down to a binary classification task, i.e., predicting whether a node v is under attack (class 1) or not (class 0), belonging to the node-level prediction.

For a node v in graph G (to distinguish from the attack graph $\mathcal{G}$ introduced in section 3.2), GNN learns a hidden representation, $\mathbf{h}_v$, by taking both the node features $\mathbf{x}_v$ and graph structure (edges) E as input. Most of the advanced GNNs follow a message-passing and aggregation strategy, in which the information of one node is iteratively passed to the others along edges, and then the node's representation is updated by aggregating and learning the information of itself and its neighbors. An example of such a message-passing and aggregation strategy is depicted in Figure 4. When conducting the message propagation with l-layered GNN, one node could gain the information of neighboring nodes within l hops. Without loss of generality, the latent representation at the l-th layer of GNN can be formulated as:

$$\mathbf{h}^l_v = f^l(\mathbf{h}^{l-1}_v, AGG^l(\{\mathbf{h}^{l-1}_u, \forall u \in \mathcal{N}_v\})) \quad (3)$$

where $\mathcal{N}_v$ refers to the neighboring nodes of node v. AGG^l refers to aggregations that could be rather simple ones (e.g., mean, max, sum), to advanced ones (e.g., median, var, std), or even learnable

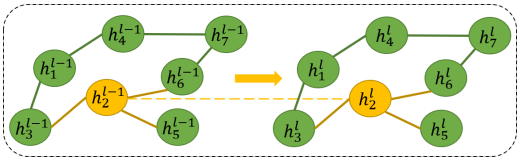


Figure 4: Diagram of message passing and aggregation in GNN. For node 2, we have $h_2^l = (h_2^{l-1}, h_3^{l-1}, h_5^{l-1}, h_6^{l-1})$.

and exotic ones. f^l is a function with learnable parameters, e.g., multi-layer perceptron (MLP). The combination of AGG^l and f^l determines the types of graph operators, e.g., graph convolutional operator and graph attentional operator.

For node-level prediction, the output of the last hidden layer could be directly passed to the output layer Φ, e.g., a fully connected layer. For graph-level prediction, it is passed to a readout function R with learnable parameters. Formally, the representations of the output layer can be expressed as:

$$\mathbf{h}_v = \Phi(\mathbf{h}_v^l) \tag{4}$$

$$\mathbf{h}_G = R(\mathbf{h}_v^l | v \in V) \tag{5}$$

Graph convolutional operator. Graph convolution, essentially the same as the convolution for image classification task, is a generalization of convolution in Convolutional Neural Network (CNN) in which the information of neighboring nodes (corresponding to adjacent pixels) is convoluted with that of central node (corresponding to central pixel) to learn a hidden representation. The graph convolution operation in [22] could be formulated as:

$$\mathbf{h}_i' = \sigma(\mathbf{W}^T \sum_{j \in \mathcal{N}_i \cup \{i\}} \frac{w_{ji}}{\sqrt{\hat{d}_j \hat{d}_i}} \mathbf{h}_j) \tag{6}$$

where $\hat{d}_i = 1 + \sum_{j \in \mathcal{N}_i} w_{ji}$. $w_{ji} \in \mathbb{R}$ denotes the weight of edge from source node j to target node i. $\mathbf{W}$ is the filter weight to be optimized. σ is the activation function. T refers to transposition. By default, all edge weights are fixed as 1, meaning neighboring nodes are of the same importance to central node. Edge weights can also be optimized to learn the importance of neighboring nodes in an implicit manner.

Graph attentional operator. In [37], an attention mechanism is introduced to learn the edge weights between two connected nodes, in which the connective strength α_{ij} is explicitly calculated as a self-attention for the neighborhoods, formulated as:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}_i} \exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_k]))} \tag{7}$$

where $\mathbf{a}$ and $\mathbf{W}$ are learnable parameters and $\parallel$ denotes concatenation. Then, the hidden representation for node i can be obtained by learning the weighted hidden representations of neighboring nodes:

$$\mathbf{h}_i' = \sigma(\sum_{j \in \mathcal{N}_i} \alpha_{ij} \mathbf{W}\mathbf{h}_j) \tag{8}$$

The attention mechanism can be extended with K-head attention:

$$\mathbf{h}_i' = \|_{k=1}^K \sigma(\sum_{j \in \mathcal{N}_i} \alpha_{ij}^k \mathbf{W}^k \mathbf{h}_j) \tag{9}$$

Input graph to GNN. As desired, the inputs to GNN models should be graph-structured data. Hence, we need to construct graphs with attack graph and real-time measurements that can be fed into and learned by GNN models. In the proposed GNN-IDS, we assume the measurements are action-level, while not all nodes in attack graph have action information. Taking into account the scenario where an attack happens in attack graph that an attack usually starts from a fact node or privilege node, and passes through an action node if preconditions are satisfied, and ends at a privilege node where assets are compromised, we therefore choose to attach the measurements of a service to the privilege nodes that include the service's identity. For fact nodes and actions nodes without attached measurements, zero-padding is applied to align the lengths of all nodes. In this way, the nodes in input graphs contain both real-time measurements and attack scenarios. The computer network characterized by static attributes and dynamic attributes can then be expressed as a spatial-temporal graph in the format $G^t = (V, E, \mathbf{X}^t)$, where $\mathbf{X}^t = (\mathbf{F}, \mathbf{M}^t)$. For the proposed GNN-IDS framework, we have:

$$\mathbf{x}_v^t = \begin{cases} \mathbf{f}_v \parallel \mathbf{m}_{action}^t & \text{if } action \in S_v \text{ and } v \in \{\text{privilege nodes}\} \\ \mathbf{f}_v \parallel \{\mathbf{0} \in \mathbb{R}^K\} & \text{otherwise} \end{cases} \tag{10}$$

where $\parallel$ denotes the concatenation operator, for concatenating unchangeable features with changeable features over time. Then, we could take $\mathbf{x}_v^t \in \mathbb{R}^{D+K}$ as a snapshot of the network status at the time t and feed it to GNN models.

3.5 Inference

Generally, intrusion detection is a classification task, which could be either classifying input samples as benign or malicious (binary classification), or in a fine-grained way predicting specific attack scenarios (multi-class classification). In this paper, the output of GNN model is probabilities on which node in an input graph is positive or negative, that is, which host in the network is under attack and which action is taken to realize the attack. As one of our main contributions, we propose to analyze the predictions of GNN models from aspects of uncertainty and explainability.

Uncertainty. Uncertainty becomes a nontrivial issue for machine learning based IDSs when it comes to sophisticated attackers, who can mimick normal behaviors to evade detection. In this paper, we calculate the probabilities for benign and malicious classes for each node in input graphs and use the probabilities to reveal the confidence level of GNN models. In addition, we also evaluate the uncertainty of GNN models over different probability thresholds for separating benign and malicious samples.

Explainability. For intrusion detection, it is crucial to not only train a machine learning model with high certainty but also understand why it makes such a prediction given a sample. The GNN models, aware of network topology and optimized accordingly, can give a quantitative evaluation of which neighboring nodes or features are contributing more to the prediction. In our work, we

utilize *GNNEXPLAINER* [43] to evaluate the importance of neighboring nodes and node features. More importantly, in GNN-IDS, the network information such as attack actions and vulnerabilities is included in node representations. Thus, when an anomaly is detected, the attack action causing it is also identified, as well as the whole attack scenario. In this way, the predictions are more explainable and GNN models are more trustworthy.

3.6 Deployment of GNN-IDS

For security operators unfamiliar with this framework or even practitioners in the security domain without much security knowledge, they are also able to set up such a GNN-IDS for their local computer networks. The flow to develop and deploy a GNN-IDS for computer networks includes five steps below:

(1) **Information gathering.** The first step, prerequisite to step 2, is to collect the network information for attack graph generation, e.g., hosts, network configurations, assets, vulnerabilities, which could be obtained using network or vulnerability scanning tools.
(2) **Attack graph generation.** In the proposed GNN-IDS, we leverage MulVAL [30] to generate the attack graph corresponding to specific networks. Other approaches or toolkits for generating attack graphs could also used, only if real-time measurements and attack graph could be well associated and fed into GNN models.
(3) **Measurement capturing.** In this step, we collect the real-time data generated in the network and save them in the format of graph, and then label each sample with detailed attack scenarios (for multi-class classification) or benign/anomaly labels (for binary classification).
(4) **Association of attack graph and measurements.** In this step, we associate the captured measurements with the attack graph to build spatial-temporal graphs that could be fed into GNN models.
(5) **Model training and evaluation.** Input the spatial-temporal graphs to GNN model for training. Evaluate the model's performance and then deploy the GNN-IDS in real-world systems.

4 CASE STUDY AND DATASETS

In this section, we introduce a use case network studied in our work and discuss how to simulate the benign and attack scenarios and prepare the datasets containing real-time measurements.

4.1 Use Case Network

Network information. To study the effectiveness of GNN-IDS, we borrow a small-sized but real-world network from [30], whose architecture is depicted in Figure 5. In the network, the Internet Zone, Demilitarized Zone (DMZ), and Internal Zone are separated by external and internal firewalls. The web server *webServer* is within the DMZ. The servers *fileServer* and *workStation* are located in the Internal Zone. All servers are managed by administrators. Common users could use *workStation* to run an executable stored in *fileServer* for computation if authenticated. The vulnerabilities [8] present in the network are collected in Table 1.

Attack scenarios. First, an attacker who has internet access remotely exploits vulnerability *CVE-2002-0392* to gain local access to *webServer*. Next, the attacker exploits vulnerability *CVE-2003-0252* on *fileServer* to gain root privilege. Alternatively, the attacker

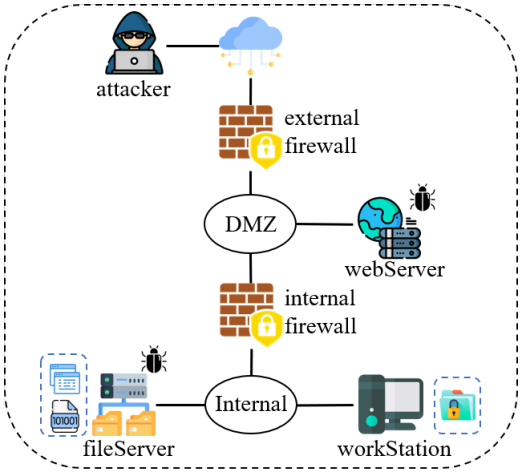


Figure 5: Architecture of use case network. The *attacker* located on the internet could steal valuable data on *workStation* by exploiting the vulnerabilities on *webServer* and *fileServer*.

could modify files on *fileServer* through NFS protocol if the NFS export table is incorrectly configured. After escalating privilege, the attacker can install a Trojan-horse malware in the executable binaries on *fileServer*. As common users frequently use *workStation* for computation, it is very likely that the Trojan-horse is mounted and executed on *workStation*, causing severe data leakage issues.

Encoded attack graph. With the gathered network information above, we can generate an attack graph for the use case network. Next, We parse the node statements in the attack graph to extract tokens and save them into a corpus. We then encode the attack graph by following the Algorithm 1. For the use case network, we obtain a matrix in which elements are either 0 or 1 representing the node statements, as well as a list of pairs representing the node connectivity in the attack graph.

4.2 Dataset Preparation

In this paper, we prepare two datasets (Dataset 1 and Dataset 2) specific to the use case network for evaluation. Dataset 1 is an artificial dataset in which the distribution of measurements can be well controlled to simulate complex attack and benign scenarios. Dataset 2 is synthetically created using the network traffic samples in public CIC-IDS2017 dataset to simulate more realistic attack and benign behaviors. We consider both datasets in our work because they complement each other in complexity and veracity.

Dataset 1. Following the assumptions in section 3.3, we assume 78 (equal to the feature size of CIC-IDS2017) measurements for the use case network. Among these measurements, every three comprise a group in which the first follows Gaussian Distribution, the second follows Poisson Distribution, and the third depends on the first two. In the generated attack graph, there are 7 privilege nodes corresponding to non-persistent or persistent actions, meaning that we have 7 attack scenarios and 1 benign scenario to simulate. The sampling rules for a certain group of measurements are shown in Table 2. Each input graph, a snapshot of the network's status that

Table 1: Vulnerabilities in the use case network

Server	CVE ID	Impacted Protocol	Description
webServer	CVE-2002-0392	HTTP	Allow remote execution of code or DoS attack
fileServer	CVE-2003-0252	NFS	Allow remote execution of code or DoS attack

is producing 78 measurements, is the realization of either one of the 7 attack scenarios or benign scenario. For Dataset 1, in total, we have 5000 input graphs in our collection, of which $500 \times 7 = 3500$ are attack scenarios and 1500 are benign scenario. In each graph realizing attack scenarios, the compromised privilege node is labeled as 1 and the other 6 are labeled as 0. In each graph realizing benign scenario, all the 7 privilege nodes are labeled as 0. It should be noted that Dataset 1 contains imbalanced benign nodes (90%) and malicious nodes (10%), which is normal in practice.

Dataset 2. In the use case network, there are 7 privilege nodes that the attacker could compromise step by step. We treat each compromise as an attack scenario in CIC-IDS2017, and then associate the privilege nodes with attacks (Web Attack - Brute Force, DoS slowloris, FTP-Patator, SSH-Patator, DDoS, Bot, PortScan) in CIC-IDS2017. For each attack scenario, we take 500 snapshots of network status and store them in graph format. Same as Dataset 1, Dataset 2 also contains 5000 input graphs, of which 3500 are the realizations of the 7 attack scenarios and the other 1500 are the realizations of benign scenario. Similarly, there is only one malicious node in each input graph realizing attack scenarios. In total, we have 3500 malicious nodes (labeled as 1) and 31500 benign nodes (labeled as 0). At the node-level, the anomaly rate of Dataset 2 is 10%, which is comparable to that of CIC-IDS2017.

5 EXPERIMENTAL RESULTS

5.1 Experiment Setups

Execution environment. All experiments are conducted on the Dell XPS 15 9520 laptop with the 12th Gen Intel Core i7-12700H CPU (14 cores), 32GB installed RAM, and Nvidia GeForce RTX 3050 Ti Laptop GPU (4GB memory). The source codes are implemented under *Python* environment (version '3.8.10') and the GNN models are built on the graph operators in *torch_geometric* (version '2.3.1') in *PyTorch* (version '2.0.1+cu117').

Datasets. We first re-scale the data in Datasets 1 and 2 using the min-max normalization, and then split each into three subsets for training, validation, and test purposes in a stratified way, ensuring that they are identically distributed. For both Datasets 1 and 2, we have 3000 input graphs for training, 1000 for validation, and the other 1000 for test. In our proposed GNN-IDS, the intrusive behaviors are detected at node-level. Each input graph to GNN models contains 7 vulnerable nodes, out of which at most one is considered compromised at a certain time point. Consequently, we have 21000 nodes for training, 7000 for validation, and 7000 for test.

Models. From the attack graph generated by MulVAL, we observe that a certain attack starts from a fact node or privilege node and passes through an action node, and ends up at another privilege node. To enable GNN models to learn complete attack scenarios, we set the number of hidden layers to 2. The architectures of GNN models are presented in Table 3, which also includes classical neural network (NN) model that is taken as the baseline. It is worth noting that the inputs to NN model are individual samples, while the inputs to GNN models are graph-structured data.

Hyper-parameters. In Datasets 1 and 2, we have imbalanced benign samples (90%) and malicious samples (10%), which is common in real-world systems as attacks rarely happen. To trade off the recall and precision, we add weight to the positive (malicious) samples when calculating cross-entropy loss. The weight equals to the size of negative samples over that of positive ones. We use the *Adam* optimizer (learning rate $1.0e-3$) and mini-batch (batch size 30) for training. For fair performance evaluations, we use early-stopping if overfitting on the validation set is observed.

Evaluation metrics. In our experiments, standard metrics [40] such as Precision, Recall, F1-score, False Positive Rate (FPR), False Negative Rate (FNR), are used for evaluating detection accuracy. Additionally, the Receiver Operating Characteristics (ROC) curve representing the prediction probability and Area Under ROC curve (AUC) representing the measure of separability are used for uncertainty evaluation.

5.2 Results and Analysis

To the best of our knowledge, there are no state-of-the-art approaches that leverage both attack graph and real-time measurements for intrusion detection and can be used for performance comparisons. We therefore take classical NN models as the baseline in this paper, which is common in many ML-based IDSs [36]. To validate the effectiveness and efficiency of GNN-IDS, three different GNN models, i.e., GCN, GCN-EW, and GAT, are developed and evaluated. The results are presented and analyzed from three aspects, i.e., uncertainty, explainablility, and robustness, corresponding to the three research questions in section 1.

Uncertainty. The standard metrics of NN and GNN models evaluated on Datasets 1 and 2 are collected in Table 4, in which the best are highlighted in bold. We could see the GNN models outperform NN model on all standard metrics for Dataset 1. For Dataset 2, GNN models achieve superior performance, especially in Precision, F1-score, and FPR. The advance of GNN can be visually observed in Figure 6, in which False Positives are effectively mitigated by GNN models. In Figure 7, we plot the ROC curves of NN and GNN models. For Dataset 1, we observe the AUC of GNN models increases by around 5%, indicating GNN models are more confident in their predictions. For Dataset 2, the ROC curves show NN and GNN models achieve comparable separability and uncertainty. The reason is that the benign and malicious samples in Dataset 2 are well separated from each other in the original feature space, making the patterns easy to extract for both NN and GNN models.

Table 2: Sampling rules (for three measurements) for simulation of benign and malicious scenarios in Dataset 1

Scenario	No. Input Graphs	$m_1 \sim \mathcal{N}(\mu, \sigma)$	$m_2 \sim \mathcal{P}(\lambda)$	$m_3 = (m_1, m_2)$
benign	1500	$\mu = 0.3, \sigma = 0.2$	$\lambda = 3.0$	$\sqrt{\lvert m_1 \rvert} + 0.5m_2 + 0.5$
attack (non-persistent action)	500	$\mu = 0.3, \sigma = 0.2$	$\lambda \sim U(1.0, 5.0)$	$\sqrt{\lvert m_1 \rvert} + 0.5m_2 + 0.5$
attack (persistent action)	500	$\mu \sim U(0.1, 0.5), \sigma = 0.1$	$\lambda = 3.0$	$\sqrt{\lvert m_1 \rvert} + 0.5m_2 + 0.5$

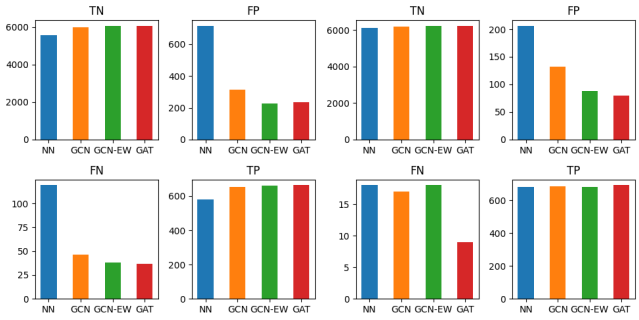
Table 3: Architectures of NN and GNN models

Model	Hidden Layer 1	Hidden Layer 2	Output Layer
NN	ReLU(Linear(K, 20))	ReLU(Linear(20, 20))	Sigmoid(Linear(20, 1))
GCN	ReLU(GCNConv(D+K, 20))	ReLU(GCNConv(20, 20))	Sigmoid(Linear(20, 1))
GCN-EW	ReLU(GCNConv(D+K, 20))	ReLU(GCNConv(20, 20))	Sigmoid(Linear(20, 1))
GAT	ReLU(GATConv(D+K, 20, heads=4))	ReLU(GATConv(80, 20, heads=4))	Sigmoid(Linear(80, 1))

K: feature size of real-time measurements. D: feature size of node statements in attack graph.

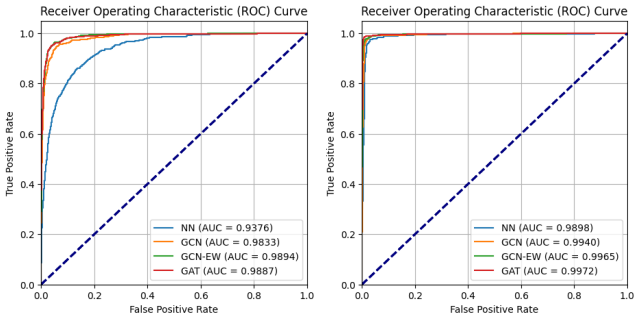
Table 4: Statistics on standard metrics (in percentage) of NN and GNN models evaluated on Datasets 1 and 2

Dataset	Model	Prec	Recall	F1-score	AUC	FPR
1	NN	71.35	85.82	75.62	93.76	11.37
	GCN	83.47	94.24	87.80	98.33	4.95
	GCN-EW	**86.84**	95.47	**90.53**	**98.94**	**3.63**
	GAT	86.61	**95.49**	90.39	98.87	3.73
2	NN	88.25	97.08	92.04	98.98	3.27
	GCN	91.76	97.74	94.49	99.40	2.10
	GCN-EW	94.14	98.02	95.97	99.65	1.40
	GAT	**94.74**	**98.72**	**96.62**	**99.72**	**1.27**



(a) Dataset 1 **(b) Dataset 2**

Figure 6: Counts of True Negatives (TN), False Positives (FP), False Negatives (FN), and True Positives (TP) on the test set of Datasets 1 and 2. Left: Dataset 1. Right: Dataset 2.



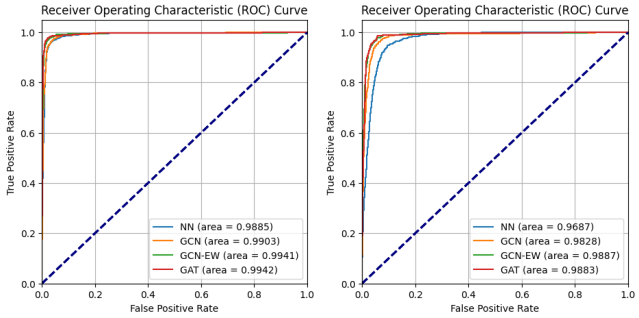
(a) ROC curves on Dataset 1 **(b) ROC curves on Dataset 2**

Figure 7: ROC curves evaluated on the test set of Datasets 1 and 2. Left: Dataset 1. Right: Dataset 2.

To further analyze the uncertainty of GNN-IDS, we created two variations of Dataset 2 (Dataset 2-v1 and Dataset 2-v2) by adding Gaussian noise with a mean value 0 and a standard deviation 0.01 and 0.05, respectively. We use the two variations to re-train the models and plot the ROC curves in Figure 8. We could see that the areas under the ROC curves of NN and GNN models are smaller, but the differences get larger which can be observed more intuitively for Dataset 2-v2. Evidently, GNN models perform better at uncertainty than NN model for both variations of Dataset 2.

It turns out that the GNN models achieve superior performance at detection accuracy. More importantly, they are more confident in the predicting results, i.e., how likely the input samples are malicious or benign. This is because GNN models can obtain a better representation in latent space by learning the information of central node and its neighboring nodes. In addition, in our proposed GNN-IDS, the GNN models learn complete attack scenarios to detect anomalies and can gain more confidence to anomalous samples.

Explainability. It is essential to understand why an IDS predicts a sample as malicious or benign. Through the incorporation of an attack graph, the proposed GNN-IDS can not only detect which node is under attack, but also present by which action the node is attacked. Besides, GNN models can also learn the importance of neighbors and node attributes by extracting patterns of inter-dependencies of network. In this paper, we rely on *GNNEXPLAINER* [43] to make quantitative and qualitative analysis to evaluate which



(a) ROC curves on Dataset 2-v1 **(b) ROC curves on Dataset 2-v2**

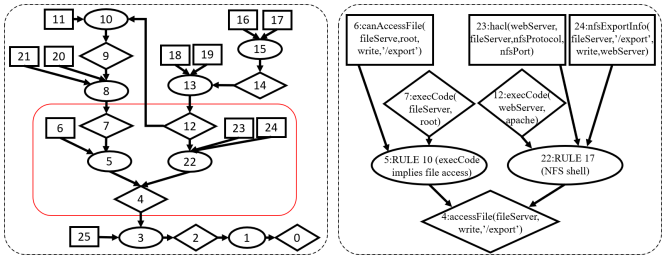
Figure 8: ROC curves evaluated on the test set of two variations of Dataset 2. Left: Dataset 2-v1, a variation of Dataset 2 with the Gaussian noise $\mathcal{N}(0, 0.01)$. Right: Dataset 2-v2, a variation of Dataset 2 with the Gaussian noise $\mathcal{N}(0, 0.05)$.

node features are more important and which edges (neighbors) are contributing more to predictions.

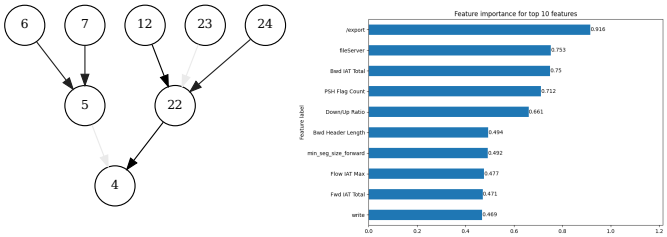
In Figure 9, we showcase the explanation for a certain prediction by GAT model. The entire attack graph is shown in Fig. 9a, in which an attack sub-graph is marked in red rectangle and detailed in Fig. 9b. In the attack sub-graph, the node with identity 4 is predicted as malicious by the GAT model, indicating that the attacker is writing malicious contents into the files under the directory '*/export*' on *fileServer*. This attack may be conducted by taking the left path (node 5) or the right one (node 22). The importance of edges (neighbors) that contribute to the prediction is shown in Fig. 9c (the arrow opacity represents the edge importance) and the top 10 node features (the bar length is proportional to the importance of features) are plotted in Fig. 9d. From the importance of both nodes and features, we could see which neighboring nodes of node 4 and which node attributes have more influence on the prediction. They are informative for system experts to make concrete diagnostics to the specific attacks. Of the top 10 features, '*fileServer*', '*/export*', and '*accessFile*' are features in the attack graph, and the others are extracted from real-time measurements, i.e., the network traffic.

To make more comprehensive analyses on the explainability of GNN-IDS , we calculate the average of feature importance over all samples in the test set of Dataset 2 for each attack scenario. For the evaluation of each sample, we record the top 10 most important features and corresponding importance. The occurrence frequencies of the recorded features at different levels of importance are then obtained, where the feature importance is descendent from level 0 to level 9. The average feature importance for node 4 on Dataset 2 and the feature distribution at different levels of importance are shown in Figure 10, in which the top 5 features with the largest occurrences at each level of importance are plotted. In the test set of Dataset 2, we have 100 malicious input graphs for the attack scenario of node 4 under attack.

From Figure 10, we could see that features are either from the attack graph or the network traffic, indicating that GNN models can learn the static and dynamic features, simultaneously, and quantify their influence on the predictions. The features with high importance could provide intuitive and valuable information to



(a) Attack graph **(b) Attack sub-graph**



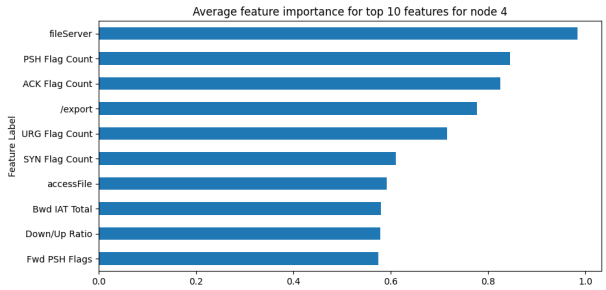
(c) Node importance **(d) Feature importance**

Figure 9: Showcase of explanation for a certain prediction by GAT model. Fig. 9a: entire attack graph of the use case network. Fig. 9b: attack sub-graph for analyzing attacks on node 4. Fig. 9c: node importance in a case of node 4 is under attack, the opacity of arrows represents the importance level of edges (neighboring nodes). Fig. 9d: feature importance in the case of node 4 is under attack, the length of bars is proportional to the importance of features.

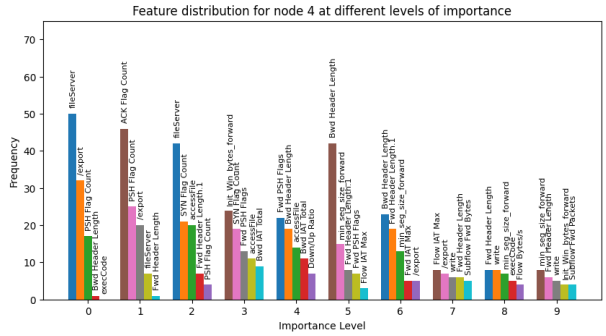
security experts, assisting them to analyze the specific intrusions. This greatly increases the efficiency of security diagnostics by experts when alerts are triggered and spread out, reducing the time to response to the attacks. We could also see the difference among the frequencies of features at each level of importance gets smaller over importance level, meaning that the contributions of features to predictions at a higher level of importance are more differentiated. Intuitively, it shows the features at higher levels of importance are more informative to analyze attack scenarios.

It turns out, the proposed GNN-IDS could make more explainable predictions by learning the static features in attack graph and dynamic features in real-time measurements, simultaneously, which is realized by associating the attack graph and real-time measurements.

Robustness. Real-world systems could evolve over time, resulting in inaccurate measurements captured from the systems. Attackers could also adapt their attacking behaviors to evade detection. To validate the effectiveness of GNN-IDS to unseen inputs, We attempt to simulate the changes in networks by introducing Gaussian noise in the malicious samples of original test set. The trained NN and GNN models are re-evaluated on the noisy test set. The performance difference between the original test set and the noisy test set are presented in Table 5. We could see the GNN models achieved smaller



(a) Average feature importance



(b) Feature distribution

Figure 10: Statistics on the feature importance and feature distribution. Fig. 10a: average feature importance for top 10 **features for node** 4 **under attack, which is evaluated on the** 100 **samples in the test set of Dataset 2. Fig. 10b: the feature frequencies of top** 5 **features with most occurrences at each level of importance.**

differences than NN model, indicating GNN models are more robust to the changes in networks. Specifically, GAT and GCN models achieve the smallest performance change for Datasets 1 and 2, respectively. More importantly, GNN models produce fewer false positives than NN model, indicating GNN models can effectively avoid detection evasions.

From the above results, we can reason that GNN models can learn a more stable hidden representation in latent space by fetching and aggregating the information of both central node and neighbors, so that the change of hidden representation caused by inaccurate measurements or sophisticated attacks is effectively mitigated. Above all, the proposed GNN-IDS is more robust to evolving networks by learning the topological attributes of networks.

Thus, the answers to our research questions are affirmative; the proposed GNN-IDS exhibits greater certainty, explainability and robustness in predictions compared to contemporary approaches.

6 CONCLUSIONS AND DISCUSSIONS

We present the GNN-IDS, a graph neural network based intrusion detection system, in which attack graph and real-time measurements are incorporated to represent the complex computer networks. Three GNN models, i.e., GCN, GCN-EW, and GAT, are developed and two synthetic datasets (one from the public dataset CIC-IDS2017) are created for evaluation. Experimental results show that, the proposed GNN-IDS achieves higher confidences at its predictions by learning structural information and node representation in networks. Moreover, due to the awareness of network information such as assets and vulnerabilities, GNN-IDS can learn more complete attack scenarios and make more explainable predictions. Last but not least, our GNN-IDS behaves robustly to evolving networks, avoiding detection evasions.

Table 5: Performance changes (in percentage) of NN and GNN models evaluated on noisy test set of Datasets 1 and 2

Dataset	Model	Prec	Recall	F1-score	AUC	FNR
1	NN	-1.03	-2.43	-1.36	-2.15	+4.86
	GCN	-0.56	-1.31	-0.81	-0.30	+2.57
	GCN-EW	-0.65	-2.00	-1.15	-0.48	+4.00
	GAT	**-0.39**	**-1.21**	**-0.69**	**-0.22**	**+2.42**
2	NN	-1.32	-4.29	-2.47	-1.25	+8.57
	GCN	**-0.25**	**-0.80**	**-0.48**	**-0.13**	**+1.57**
	GCN-EW	-0.31	-1.43	-0.826	-0.19	+2.86
	GAT	-0.63	-3.00	-1.72	-0.29	+6.00

In comparison to the start-of-the-art graph-based approaches, we explore the feasibility of applying attack graph in intrusion detection. Attack graph, composed of nodes representing static attributes and edges representing network connectivity, is the abundant information resources, but is not well utilized for intrusion detection. In general, attack graph and real-time measurements are taken as two separate techniques for different research purposes. We propose to associate the attack graph with real-time measurements to construct the input graphs that can be fed into GNN models. Through extracting features from the attack graph and real-time measurements, GNN models can learn the static, dynamic, and even topological attributes of networks, simultaneously.

For modern computer networks, it is very common that they are purposely updated to suit new computing contexts. In our work, the structural evolution of networks and the intra-dependencies between sequential input graphs are not studied. We argue that the structural evolution of computer networks needs to be tracked and analyzed to design more adaptive intrusion detection systems. This will be the focus of our next project. Furthermore, as attackers usually take sequential actions to compromise a system, the temporal relations among time series data need to be deeply investigated. To set up a small network to capture realistic data is another aspect of future works.

ACKNOWLEDGMENTS

We would like to thank our group members for their useful comments and suggestions on this work. This work has been funded by eSSENCE graduate school (https://essenceofescience.se/).

REFERENCES

[1] Leman Akoglu, Hanghang Tong, and Danai Koutra. 2015. Graph based anomaly detection and description: a survey. *Data mining and knowledge discovery* 29 (2015), 626–688.

[2] Michael L. Artz. 2002. *Netspa: A network security planning architecture.* Ph. D. Dissertation. Massachusetts Institute of Technology.

[3] Elena Basan, Maria Lapina, Nikita Mudruk, and Evgeny Abramov. 2021. Intelligent intrusion detection system for a group of UAVs. In *Advances in Swarm Intelligence: 12th International Conference, ICSI 2021, Qingdao, China, July 17–21, 2021, Proceedings, Part II 12*. Springer, 230–240.

[4] Ghanshyam S Bopche and Babu M Mehtre. 2014. Attack graph generation, visualization and analysis: issues and challenges. In *Security in Computing and Communications: Second International Symposium, SSCC 2014, Delhi, India, September 24-27, 2014. Proceedings 2*. Springer, 379–390.

[5] Amol Borkar, Akshay Donode, and Anjali Kumari. 2017. A survey on Intrusion Detection System (IDS) and Internal Intrusion Detection and protection system (IIDPS). In *2017 International conference on inventive computing and informatics (ICICI)*. IEEE, 949–953.

[6] Bahadir Candan. 2023. Top 5 critical infrastructure cyberattacks. https://www.anapaya.net/blog/top-5-critical-infrastructure-cyberattacks [Accessed: 28th Feb 2024].

[7] Frank Capobianco, Rahul George, Kaiming Huang, Trent Jaeger, Srikanth Krishnamurthy, Zhiyun Qian, Mathias Payer, and Paul Yu. 2019. Employing attack graphs for intrusion detection. In *Proceedings of the New Security Paradigms Workshop*. 16–30.

[8] The MITRE Corporation. 1999-2024. Common Vulnerabilities and Exposures (CVE) Program. https://www.cve.org/. [Accessed: 28th Feb 2024].

[9] Ailin Deng and Bryan Hooi. 2021. Graph neural network-based anomaly detection in multivariate time series. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 4027–4035.

[10] Hamdi Friji, Alexis Olivereau, and Mireille Sarkiss. 2023. *Efficient Network Representation for GNN-Based Intrusion Detection.* Springer Nature Switzerland, 532–554. http://dx.doi.org/10.1007/978-3-031-33488-7_20

[11] Chuanpu Fu, Qi Li, and Ke Xu. 2023. Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis. *arXiv preprint arXiv:2301.13686* (2023).

[12] Sebastian Garcia, Martin Grill, Jan Stiborek, and Alejandro Zunino. 2014. An empirical comparison of botnet detection methods. *computers & security* 45 (2014), 100–123.

[13] André Greubel, Daniela Andres, and Martin Hennecke. 2023. Analyzing Reporting on Ransomware Incidents: A Case Study. *Social Sciences* 12, 5 (2023). https://doi.org/10.3390/socsci12050265

[14] Sushil Jajodia and Steven Noel. 2009. Topological vulnerability analysis. In *Cyber situational awareness: Issues and research*. Springer, 139–154.

[15] Jianguo Jiang, Jiuming Chen, Tianbo Gu, Kim-Kwang R. Choo, Chao Liu, Min Yu, Weiqing Huang, and Prasant Mohapatra. 2019. Anomaly Detection with Graph Convolutional Networks for Insider Threat and Fraud Detection. In *MILCOM 2019 - 2019 IEEE Military Communications Conference (MILCOM)*. 109–114. https://doi.org/10.1109/MILCOM47813.2019.9020760

[16] Pontus Johnson, Robert Lagerström, and Mathias Ekstedt. 2018. A meta language for threat modeling and attack simulations. In *Proceedings of the 13th International Conference on Availability, Reliability and Security*. 1–8.

[17] Chaitanya K Joshi, Cristian Bodnar, Simon V Mathis, Taco Cohen, and Pietro Lio. 2023. On the expressive power of geometric graph neural networks. *arXiv preprint arXiv:2301.09308* (2023).

[18] Veeramreddy Jyothsna and Koneti M. Prasad. 2019. Anomaly-based intrusion detection system. *Computer and Network Security* (2019), 35.

[19] Ansam Khraisat, Iqbal Gondal, Peter Vamplew, and Joarder Kamruzzaman. 2019. Survey of intrusion detection systems: techniques, datasets and challenges. *Cybersecurity* 2, 1 (2019), 1–22.

[20] Dong S. Kim, Ha-Nam Nguyen, and Jong S. Park. 2005. Genetic algorithm to improve SVM based network intrusion detection system. In *19th International Conference on Advanced Information Networking and Applications (AINA'05) Volume 1 (AINA papers)*, Vol. 2. IEEE, 155–158.

[21] Hwan Kim, Byung S. Lee, Won-Yong Shin, and Sungsu Lim. 2022. Graph Anomaly Detection With Graph Neural Networks: Current Status and Challenges. *IEEE Access* 10 (2022), 111820–111829. https://api.semanticscholar.org/CorpusID:252595683

[22] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).

[23] Patrice Kisanga, Isaac Woungang, Issa Traore, and Glaucio HS Carvalho. 2023. Network Anomaly Detection Using a Graph Neural Network. In *2023 International Conference on Computing, Networking and Communications (ICNC)*. IEEE, 61–65.

[24] Sofiane Lagraa, Jérôme François, Abdelkader Lahmadi, Marine Miner, Christian Hammerschmidt, and Radu State. 2017. BotGM: Unsupervised graph mining to detect botnets in traffic flows. In *2017 1st Cyber Security in Networking Conference (CSNet)*. IEEE, 1–8.

[25] Bishal Lakha, Sara L. Mount, Edoardo Serra, and Alfredo Cuzzocrea. 2022. Anomaly Detection in Cybersecurity Events Through Graph Neural Network and Transformer Based Model: A Case Study with BETH Dataset. In *2022 IEEE International Conference on Big Data (Big Data)*. 5756–5764. https://doi.org/10.1109/BigData55660.2022.10020336

[26] Harjinder S. Lallie, Kurt Debattista, and Jay Bal. 2020. A review of attack graph and attack tree visual syntax in cyber security. *Computer Science Review* 35 (2020), 100219.

[27] Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. 2015. Gated graph sequence neural networks. *arXiv preprint arXiv:1511.05493* (2015).

[28] Wai Weng Lo, Siamak Layeghy, Mohanad Sarhan, Marcus Gallagher, and Marius Portmann. 2022. E-graphsage: A graph neural network based intrusion detection system for iot. In *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 1–9.

[29] Aleksandar Milenkoski, Marco Vieira, Samuel Kounev, Alberto Avritzer, and Bryan D Payne. 2015. Evaluating computer intrusion detection systems: A survey of common practices. *ACM Computing Surveys (CSUR)* 48, 1 (2015), 1–41.

[30] Xinming Ou, Sudhakar Govindavajhala, Andrew W Appel, et al. 2005. MulVAL: A logic-based network security analyzer.. In *USENIX security symposium*, Vol. 8. Baltimore, MD, 113–128.

[31] David Pujol-Perich, José Suárez-Varela, Albert Cabellos-Aparicio, and Pere Barlet-Ros. 2022. Unveiling the potential of graph neural networks for robust intrusion detection. *ACM SIGMETRICS Performance Evaluation Review* 49, 4 (2022), 111–117.

[32] Iman Sharafaldin, Arash H. Lashkari, and Ali A Ghorbani. 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp* 1 (2018), 108–116.

[33] Oleg Sheyner and Jeannette Wing. 2003. Tools for generating and analyzing attack graphs. In *International symposium on formal methods for components and objects*. Springer, 344–371.

[34] Siva S Sivatha Sindhu, Suryakumar Geetha, and Arputharaj Kannan. 2012. Decision tree based light weight intrusion detection using a wrapper approach. *Expert Systems with applications* 39, 1 (2012), 129–141.

[35] S Smys, Abul Basar, Haoxiang Wang, et al. 2020. Hybrid intrusion detection system for internet of things (IoT). *Journal of ISMAC* 2, 04 (2020), 190–199.

[36] Ankit Thakkar and Ritika Lohiya. 2022. A survey on intrusion detection system: feature selection, model, performance measures, application perspective, challenges, and future research directions. *Artificial Intelligence Review* 55, 1 (2022), 453–563.

[37] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, et al. 2017. Graph attention networks. *stat* 1050, 20 (2017), 10–48550.

[38] Ravi Vinayakumar, Mamoun Alazab, KP Soman, Prabaharan Poornachandran, Ameer Al-Nemrat, and Sitalakshmi Venkatraman. 2019. Deep learning approach for intelligent intrusion detection system. *Ieee Access* 7 (2019), 41525–41550.

[39] Minh N. Vu and My T. Thai. 2020. PGM-Explainer: Probabilistic Graphical Model Explanations for Graph Neural Networks. arXiv:2010.05788 [cs.LG]

[40] Ž Vujović et al. 2021. Classification model evaluation metrics. *International Journal of Advanced Computer Science and Applications* 12, 6 (2021), 599–606.

[41] Yulei Wu, Hong-Ning Dai, and Haina Tang. 2021. Graph neural networks for anomaly detection in industrial internet of things. *IEEE Internet of Things Journal* 9, 12 (2021), 9214–9231.

[42] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826* (2018).

[43] Zhitao Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. 2019. Gnnexplainer: Generating explanations for graph neural networks. *Advances in neural information processing systems* 32 (2019).